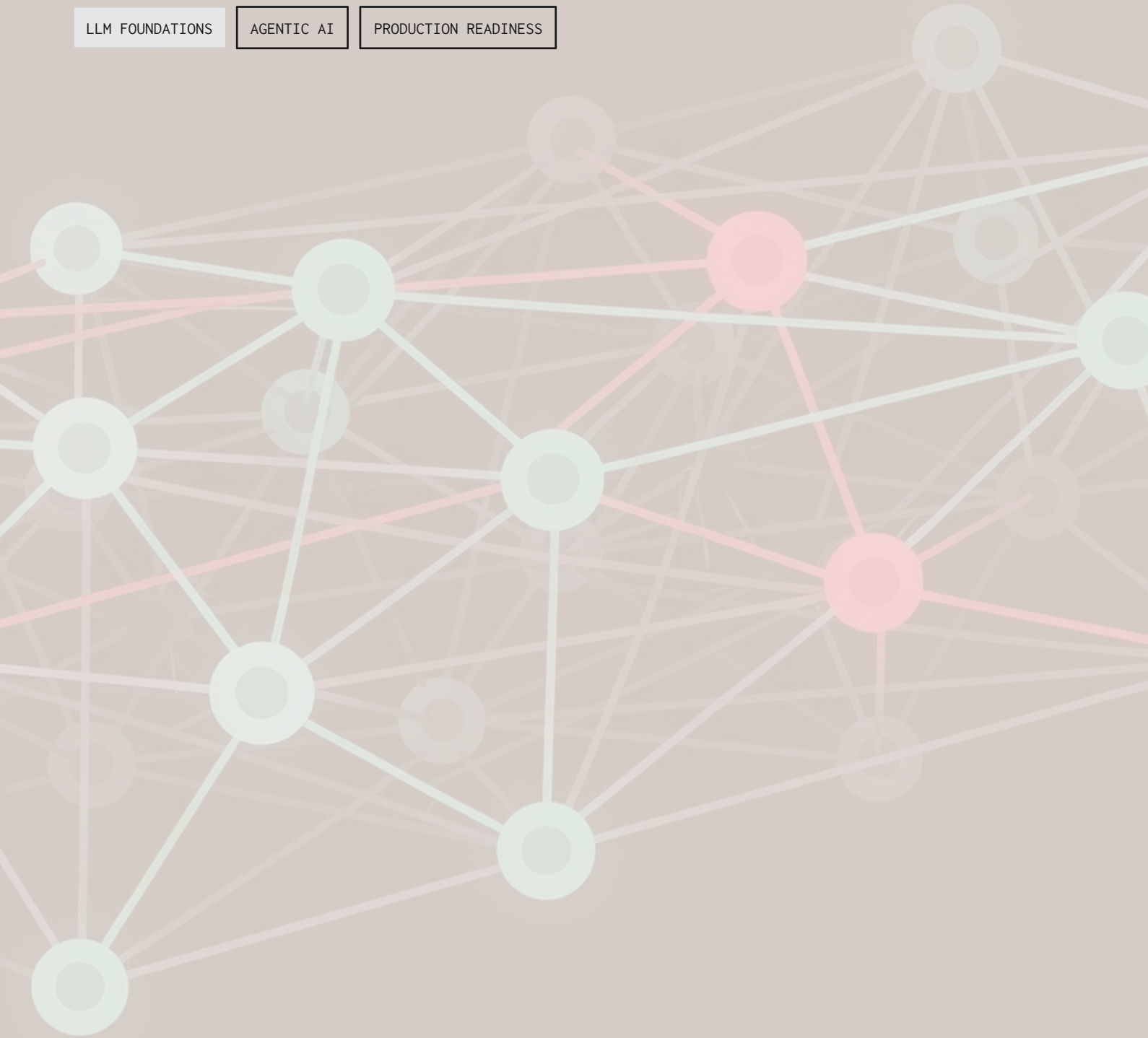
Week 1 Agentic AI Bootcamp Glossary

A concise reference for Week 1 — covering terms from the three week-one decks plus prerequisite vocabulary for building with LLMs, agents, automation tools, and AI coding harnesses.

LLM FOUNDATIONS | AGENTIC AI | PRODUCTION READINESS



LLM Foundations

Before building with language models, it's essential to understand the core concepts that define how they work, what they can do, and where they fall short.

Artificial Intelligence (AI)

Software systems that perform tasks associated with human intelligence — reasoning, language understanding, perception, planning, or decision-making. In this course, AI refers to modern model-based systems rather than hand-coded rules.

Generative AI

AI that creates new content such as text, code, images, audio, or structured data. LLMs are the text-and-code-centered branch of generative AI.

Large Language Model (LLM)

A neural network trained on large amounts of text and code to predict and generate tokens. LLMs can answer questions, write code, summarize documents, reason through tasks, and call tools when connected to an agent harness.

Foundation Model

A large pretrained model that can be adapted or prompted for many tasks. LLMs, vision-language models, and multimodal models are all examples of foundation models.

Neural Network

A machine learning model made of layers of mathematical units that learn patterns from data. Modern LLMs are very large neural networks trained with massive compute.

Parameters

The learned numerical weights inside a model. More parameters can increase capacity, but size alone does not guarantee better answers, lower cost, or better latency.

Pretraining

The broad training phase where a model learns general patterns from large datasets. For LLMs, this usually means learning to predict text across internet-scale corpora.

Fine-Tuning

Additional training that adapts a pretrained model to a narrower task, style, domain, or behavior. Fine-tuning changes the model weights, unlike prompting, which only changes the input.

Instruction Tuning

Training a model to follow human-style instructions rather than merely continue text. It helps turn raw completion models into useful assistants.

RLHF

Reinforcement Learning from Human Feedback — a training method that uses human preference data to shape model behavior. It helped make chat models more helpful, polite, and instruction-following.

Chat Model

An LLM packaged to handle role-based conversations such as system, user, and assistant messages. Chat models are usually optimized to answer instructions rather than simply complete a passage.

Completion Model

A model interface that continues a text prompt. Modern builders usually use chat or responses-style APIs instead, but completion behavior still explains how token generation works.

Non-Determinism

The property that a model may produce different valid outputs for the same prompt. This is useful for creativity, but production systems must manage it with parameters, tests, retries, and evaluation.

Hallucination

A confident but incorrect or unsupported model output. Good context, retrieval, constraints, verification, and observability reduce hallucinations but do not eliminate them.

Reasoning Model

A model optimized to spend more computation on planning, problem solving, or multi-step reasoning before answering. These models can be useful for harder tasks but may cost more and respond slower.

Extended Thinking

A product or model behavior where the model internally plans before giving a final answer. It is especially relevant for agentic workflows that require multi-step decisions.

Transformer Architecture

The transformer is the architectural backbone of virtually every modern LLM. Understanding its key components helps builders reason about model behavior, context, and capability.

1

Transformer

The neural network architecture behind most modern LLMs. It replaced recurrence with attention, making large-scale parallel training practical.

2

Attention

The mechanism that lets each token weigh which other tokens in the context are relevant. Attention is the core reason transformers can use surrounding context effectively.

3

Self-Attention

Attention applied among tokens in the same input sequence. Each token can compare itself to other tokens and pull in relevant information.

4

Attention Head

One parallel attention pathway inside a transformer layer. Multiple heads allow the model to track different kinds of relationships at the same time.

Layer

A repeated processing block in a neural network. In transformers, layers usually combine attention, feed-forward computation, normalization, and residual connections.

Embedding

A vector representation of meaning. Text, images, audio, or other data can be embedded so similar items land near each other in vector space.

Embedding Model

A model specialized for converting content into vectors. Builders use embedding models for semantic search, retrieval, clustering, recommendation, and RAG.

Vector

A list of numbers representing an item in mathematical space. In AI applications, vectors make it possible to compare meaning using distance or similarity.

Vector Space

The high-dimensional space where embeddings live. Items with related meaning tend to be closer together in this space.

Semantic Similarity

A measure of how close two pieces of content are in meaning, not just exact wording. It is central to embeddings, search, and retrieval systems.

Tokenization and Context

Tokens are the fundamental unit of LLM input and output. Understanding how text is tokenized — and how context windows work — is essential for managing cost, quality, and model behavior.

Token

The unit of text that an LLM reads and writes. Tokens may be words, parts of words, punctuation, or spaces. Both cost and context limits are measured in tokens.

Tokenizer

The component that splits text into tokens for a specific model. Different model families can tokenize the same sentence differently.

BPE

Byte Pair Encoding — a common tokenization algorithm used by GPT, Llama, Mistral, and many other models. It repeatedly merges frequent character or byte pairs into reusable token pieces.

Vocabulary

The set of tokens a model tokenizer knows. Each model family learns or defines its own vocabulary, which affects token counts and costs.

Input Tokens

Tokens sent to the model, including user messages, system prompts, retrieved context, tool results, and conversation history. Input tokens are usually cheaper and can be processed in parallel.

Output Tokens

Tokens generated by the model. Output tokens are often more expensive because they are produced one at a time.

Context Window

The maximum amount of input plus output the model can handle in one call. Anything outside the context window is invisible to the model unless reintroduced.

Working Memory

A practical way to think about the context window. The model does not remember past calls unless the application sends relevant history or stored memory back in.

Context Length

The token capacity of a model's context window. Longer context helps with large documents, but quality still depends on relevance, sufficiency, and structure.

⚠ **Lost in the Middle:** A failure pattern where models retrieve information better from the beginning or end of a long context than from the middle. Long prompts need careful structure, not just more tokens.

⊗ **Context Rot:** The degradation that happens when long context includes too much stale, irrelevant, duplicated, or poorly structured information. Better data plumbing often helps more than simply switching to a bigger model.

⊗ **Context Pollution:** Accumulated irrelevant history, old errors, abandoned approaches, or stale decisions inside a conversation. For coding and agent work, starting a fresh task context can improve quality.

Prompt and Message Design

Prompt engineering is less about magic phrases and more about precise specs, context, and evaluation. A well-structured prompt states the goal, constraints, required format, examples, and edge cases.

1

Prompt

The input instructions and context sent to a model. A good prompt states the goal, constraints, required format, examples, and edge cases when needed.

2

System Prompt

The high-priority instruction layer that defines the model's role, rules, tone, and boundaries. It is billed on every call, so long system prompts affect both cost and latency.

3

User Prompt

The instruction or request from the end user. It usually has lower priority than the system prompt but drives the task the model should complete.

4

Prompt Anatomy

The structure of a useful prompt, often including role, task, context, constraints, examples, output format, and acceptance criteria. Clear anatomy reduces ambiguity.

Few-Shot Prompting

Providing examples in the prompt so the model can infer the desired pattern. Useful for formatting, tone, classification, extraction, and domain-specific behavior.

Zero-Shot Prompting

Asking the model to perform a task without examples. Works best for simple or common tasks that the model already understands well.

Chain-of-Thought

A prompting or model behavior where reasoning steps are generated before an answer. In production, builders often prefer concise rationales or hidden reasoning-style models.

Output Format

The structure the model must return — prose, JSON, Markdown, a table, or a list. Specifying the output format is essential for automation and downstream parsing.

Structured Output

A controlled model response that conforms to a fixed schema, often JSON. Used when the response must feed another system instead of just being read by a human.

Schema

A formal description of fields, types, and constraints for structured data. In LLM apps, schemas make outputs easier to validate and use safely.

JSON

A lightweight data format made of objects, arrays, strings, numbers, booleans, and null. Function definitions, structured outputs, and many API requests use JSON.

⊗ **Prompt Anti-Pattern:** A prompting habit that makes output worse — vague asks, conflicting instructions, missing constraints, or asking for too many unrelated tasks at once. Many model failures begin as instruction failures.

> **Prompt Engineering** is the practice of designing prompts so models produce more useful and reliable outputs. It is less about magic phrases and more about precise specs, context, and evaluation.

Model APIs and Parameters

LLM APIs let your code send prompts, receive responses, stream tokens, and use tools. Understanding the key parameters and concepts helps you build reliable, cost-effective applications.

API

An application programming interface that lets software call another service. LLM APIs let your code send prompts, receive responses, stream tokens, and use tools.

API Request

The payload your application sends to a model provider. It usually includes the model name, messages, parameters, tools, and output constraints.

API Response

The provider's returned result, including generated content, tool calls, usage metadata, and sometimes safety or finish information. Production systems should inspect more than just the text.

API Key

A secret credential used to authenticate with a model provider. Treat it like an open credit card: never commit it to GitHub, expose it in client code, or share it casually.

Environment Variable

A secure way to pass configuration such as API keys into an application at runtime. It keeps secrets out of source code and makes deployments easier to manage.

Streaming

Returning generated tokens incrementally as they are produced. Streaming improves perceived speed for chat interfaces because users see the first tokens sooner.

Non-Streaming

Waiting for the full model response before returning anything. It is simpler for batch jobs, structured outputs, and workflows where partial text is not useful.

Temperature

A parameter that controls randomness in generation. Lower temperature is more consistent; higher temperature is more varied and creative.

Top-p

A sampling parameter that limits token choices to a probability mass cutoff. It is another way to tune randomness and diversity.

Max Tokens

The maximum number of output tokens the model may generate. Setting it too low can cut off answers, while setting it too high can waste money or slow responses.

Stop Sequence

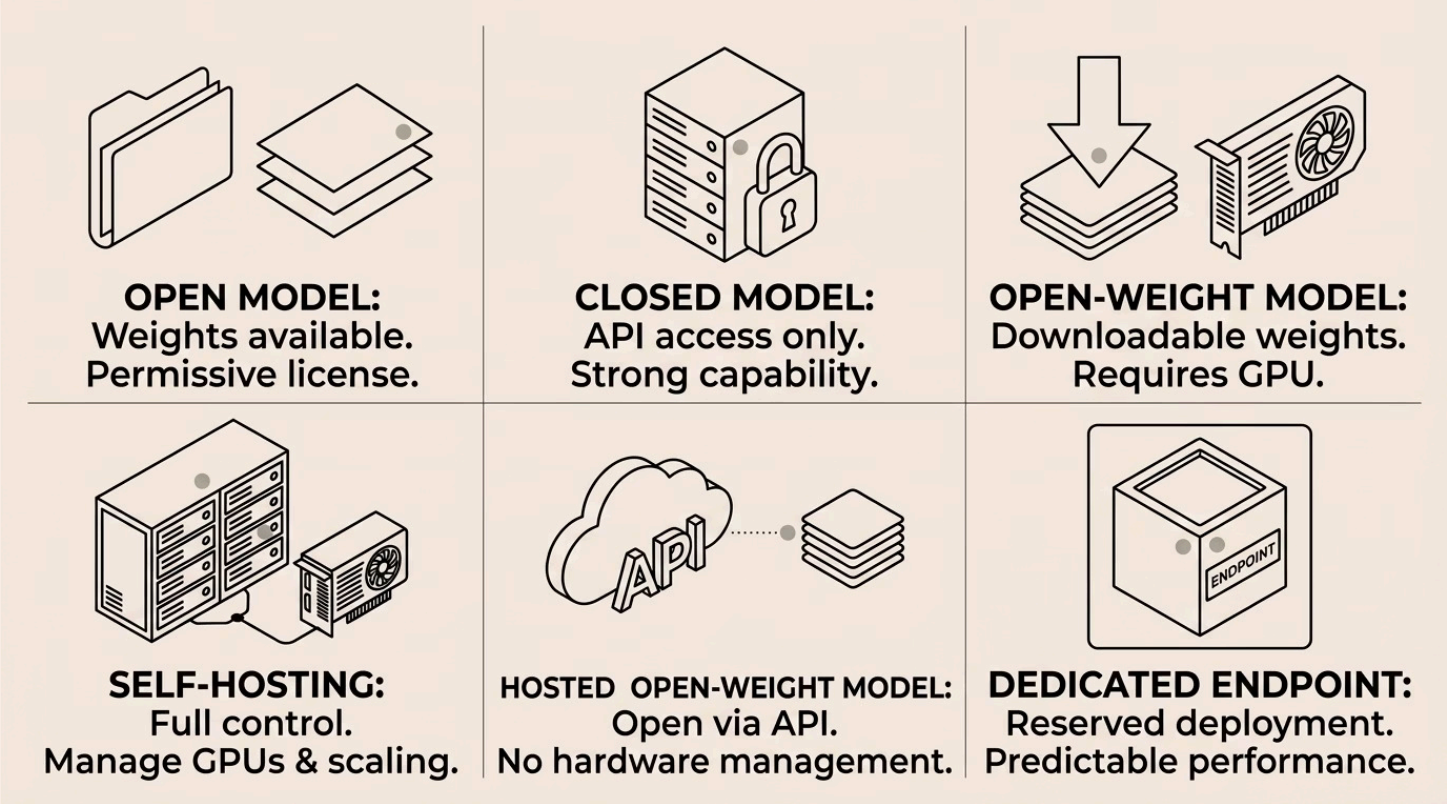
A string or token pattern that tells generation to stop. Stop sequences are useful for templates, parsers, and preventing unwanted continuation.

Rate Limit

A provider-imposed limit on how many requests or tokens you can use in a time window. Production apps need retries, backoff, fallbacks, or queueing to handle rate limits.

Model Selection and Landscape

The model landscape is vast and evolving. Builders need enough awareness of providers, open and closed models, hubs, benchmarks, pricing, and hosting options to choose wisely before committing to an architecture.



Inference Provider

A company or platform that hosts models and exposes them through APIs. Examples include Together AI, Fireworks AI, Nebius, and Groq.

Model Hub

A marketplace or repository where models are listed, documented, and shared. Hugging Face is the best-known model hub for open and open-weight models.

Hugging Face

A platform for models, datasets, demos, and AI tooling. Builders use it to discover models, compare licenses, access datasets, and try demos in Spaces.

Hugging Face Spaces

Hosted demo apps for models and AI projects. They are useful for trying behavior before setting up infrastructure.

Hugging Face Inference Providers

A marketplace-style route for calling models hosted by different providers from the Hugging Face interface. Useful for testing and comparing options quickly.

Hugging Face Inference Endpoints

Dedicated model-serving infrastructure managed through Hugging Face. Use these when you need more control, scale, or production isolation.

LM Arena

A live leaderboard based on blind A/B user preference voting. Useful for real-world preference signals but can be biased toward confident or verbose answers.

Elo Score

A ranking system originally used in chess and now used by some model leaderboards. In LM Arena, models gain or lose rating based on which response users prefer.

Static Benchmark

A fixed evaluation dataset such as MMLU or HumanEval. Static benchmarks are useful but can become stale, overfit, or misaligned with your exact use case.

OpenRouter

A unified API gateway for accessing many model providers through one interface. It helps with model swapping, price comparison, routing, and fallbacks.

Model Routing

Sending different tasks to different models based on cost, latency, capability, or reliability. Simple tasks can use cheaper models while hard tasks use frontier models.

Automatic Fallback

Switching to another model or provider when the primary path fails, times out, or hits a rate limit. It improves reliability but requires careful quality controls.

Frontier Model

A model near the current leading edge of capability. Frontier models are often more expensive but useful for complex reasoning, coding, and agentic work.

Model Trade-Offs

Choosing a model is never just about raw capability. Every deployment decision involves balancing capability, cost, latency, privacy, and vendor risk. These trade-offs should be evaluated against your specific use case.

Capability

How well a model performs on the task you care about. Capability should be judged by your use case, not by a generic overall leaderboard alone.

Cost

The money spent on input tokens, output tokens, hosting, storage, and infrastructure. Cost control requires prompt discipline, caching, routing, batching, and model choice.

Latency

The time it takes for a model system to respond. Users feel latency strongly in chat, while agents and batch workflows may care more about total completion time.

Privacy

The degree to which data is protected from exposure, logging, training use, or cross-tenant access. Privacy requirements often shape model provider, hosting, and architecture choices.

Data Residency

The rule or requirement that data stay in a specific region or jurisdiction. It matters for regulated industries, enterprise buyers, and sovereign hosting.

Vendor Lock-In

Dependency on a specific provider's API, models, tooling, or pricing. Abstractions, routing layers, and portable schemas can reduce lock-in, but not remove it entirely.

Latency Metrics and Performance

Performance in LLM systems is multidimensional. Understanding the right metrics helps you design systems that feel fast to users and scale efficiently under load.

TTFT

Time to First Token

How long it takes before the user sees the first streamed token. Strongly affects perceived responsiveness in chat interfaces.

TPS

Tokens Per Second

The speed at which output tokens stream after generation begins. Higher TPS feels smoother, especially for long answers.

E2E

End-to-End Latency

Total time from request start to final result. Matters most for agents, workflows, and batch tasks where the user waits for completion.

Throughput

How much work a system can process over time — requests per minute or tokens per second across many users. It matters for scaling beyond demos.

Batching

Combining multiple requests or operations to improve throughput or reduce overhead. Batching can lower cost but may increase individual request latency.

Caching

Reusing previous results or provider-side prompt prefixes instead of recomputing everything. Caching can reduce cost and latency when repeated context or queries are common.

Reliability and Observability

LLM reliability requires application design, not just a better model. Bad answers often look superficially fine — observability is what lets you catch and fix them systematically.

Reliability

The ability of an application to work correctly despite provider outages, rate limits, bad outputs, and unexpected inputs. LLM reliability requires application design, not just a better model.

Retry

Re-running a failed request or step. Retries should use limits, backoff, and failure classification so the system does not loop forever.

Backoff

Waiting longer between retries after failures. Backoff prevents your app from making an outage or rate-limit problem worse.

Timeout

A maximum wait time for an operation. Timeouts prevent an agent or app from hanging indefinitely when a model, tool, or API is slow.

Circuit Breaker

A reliability pattern that temporarily stops calling a failing service. It protects the rest of the app and can trigger fallbacks.

Observability

The ability to inspect what your LLM app is doing through logs, traces, metrics, and evaluations. It is essential because bad answers often look superficially fine.

Logging

Recording requests, responses, tool calls, errors, and metadata for debugging. Logs should avoid leaking secrets or sensitive user data.

Tracing

Capturing the sequence of steps in an agent or workflow. Traces help debug which prompt, model call, tool call, or decision caused a failure.

Metric

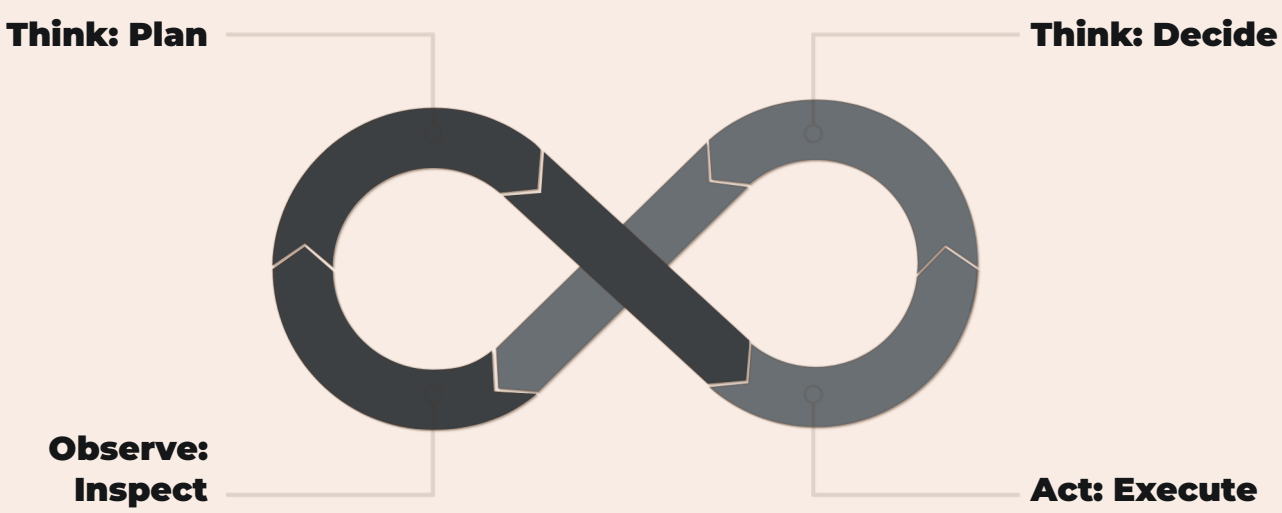
A measured signal such as cost per request, latency, success rate, tool error rate, or user satisfaction. Metrics turn vague model quality into something you can monitor.

Evaluation

A systematic way to test whether a model or app performs well. Evaluations can be automated, human-reviewed, benchmark-based, or task-specific.

Agentic AI

An AI agent can pursue a goal over multiple steps — planning, using tools, observing results, and adjusting. The model is the brain, but the harness controls the work.



The Think-Act-Observe loop is what makes a system feel agentic rather than static. Every iteration brings the agent closer to its goal — or reveals a need to replan.

AI Agent

An AI system that can pursue a goal over multiple steps, often by planning, using tools, observing results, and adjusting. The model is the brain, but the harness controls the work.

Agentic Workflow

A workflow where an LLM participates in planning or decision-making instead of only producing a one-shot response. It usually includes loops, tool calls, state, and verification.

Planning

Breaking a goal into steps before or during execution. Planning helps agents handle multi-step tasks but must be checked against reality after each action.

Tool Use

Letting a model request external actions such as search, code execution, file reads, database queries, or API calls. Tool use connects language reasoning to the outside world.

Tool Call

A structured request from the model to run a specific function or tool with arguments. The app or harness executes the tool and returns the result to the model.

Function Calling

A model capability where the LLM outputs structured arguments for predefined functions. It lets LLMs trigger actions without pretending to execute them directly.

Function Definition

The JSON-like description of a tool name, purpose, and arguments. Clear definitions help the model call the right function with valid inputs.

Agent Harness

The surrounding software that gives the model context, tools, permissions, memory, budgets, and verification. It mediates every action so the model is not operating uncontrolled.

Guardrails

Constraints that limit unsafe, expensive, incorrect, or unauthorized behavior. Guardrails can include schema validation, access control, human review, budget caps, and policy checks.

Permissions

Rules defining what an agent is allowed to access or change. Especially important for tools that read private data, spend money, send messages, or modify files.

Human in the Loop

A workflow where a human approves, reviews, or guides important agent actions. Useful for high-risk steps such as sending emails, deleting data, or deploying code.

Step Limit

A maximum number of actions an agent may take. Step limits prevent runaway loops and control cost.

Budget Cap

A spending or token limit for an agent run. Budget caps keep experimentation and automation from becoming unexpectedly expensive.

Verification

Checking that the requested work actually happened, not just that the model claimed it happened. For coding, this may mean tests; for documents, readback; for tools, inspecting returned state.

Error Recovery

Detecting and correcting failures such as tool errors, invalid arguments, bad plans, loops, or missing context. Strong agents recover deliberately instead of retrying blindly.

Memory

Stored information used across turns or sessions. Since models are stateless by default, memory must be implemented by the application through history, databases, files, or retrieval.

State

The current working information an app or agent tracks — user goal, partial results, tool outputs, and decisions. Good state management prevents agents from losing the plot.

Subagent

A specialized agent used by a main agent for a narrower task. Subagents can help parallelize work or separate responsibilities, but they add coordination complexity.

Skill

A reusable instruction or workflow package that teaches an agent how to handle a class of tasks. Skills make behavior more consistent than re-explaining the process each time.

Plugin

A packaged extension that gives an agent or app additional tools, integrations, or workflows. Plugins often connect the model to external systems.

Agentic Frameworks and Automation

Agentic frameworks provide structure so builders do not rebuild the harness from scratch. Automation tools like n8n extend these capabilities into broader workflow orchestration.

Agentic Framework

A software framework for building agents, tool calls, workflows, memory, and orchestration. Frameworks provide structure so builders do not rebuild the harness from scratch.

LangChain

A popular framework for building LLM applications with chains, prompts, tools, retrievers, agents, and integrations. Useful for prototyping and connecting many LLM app components.

LangChain Core Primitives

The basic abstractions LangChain uses — prompts, models, output parsers, retrievers, tools, and runnables. Knowing these helps you read LangChain examples without getting lost.

n8n

A workflow automation tool that connects apps, triggers, data transformations, and APIs. In agentic systems, it can orchestrate steps around an LLM or expose tools to an AI workflow.

Chain

A sequence of model calls, transformations, or tool steps. Chains are useful when the workflow is mostly predictable.

Runnable

A LangChain abstraction for something that can be invoked, composed, streamed, or batched. It helps standardize how components plug together.

Output Parser

A component that converts model text into a desired format. Parsers are often paired with structured output, schemas, or validation.

Retriever

A component that finds relevant external information for a query. Retrievers are central to RAG and document-aware applications.

01

Trigger

An event that starts an automation — a form submission, schedule, webhook, file upload, or new database row. Triggers define when work begins.

02

Workflow

A connected set of steps that move data or decisions from start to finish. Agentic workflows combine deterministic steps with LLM reasoning or generation.

03

Orchestration

Coordinating multiple steps, tools, models, and services to complete a task. Good orchestration handles order, retries, state, and error recovery.

04

Automation

Using software to execute repeated tasks with minimal human effort. Agentic automation adds model-driven decisions to workflows that were previously rule-based.

Application Mental Map

A high-level view of how an LLM app is assembled: inputs, model calls, prompts, tools, memory, data sources, outputs, evaluation, and deployment. It helps builders reason about the whole system.

LLM Application Building Block

A reusable component of an AI app — a prompt, model, retriever, tool, parser, database, queue, or evaluator. Most real apps are combinations of these blocks.

Retrieval and Data Plumbing

Many LLM app wins come from better data plumbing rather than changing models. RAG and retrieval patterns let models answer from private or current data without retraining.

RAG

Retrieval-Augmented Generation — a pattern where the app retrieves relevant external content and gives it to the model before answering. It helps models answer from private or current data without retraining.

Knowledge Base

A curated set of documents, records, or facts that an application can retrieve from. Quality depends on good ingestion, chunking, metadata, and updates.

Chunking

Splitting documents into smaller pieces for embedding and retrieval. Chunk size and boundaries affect whether the right context is found.

Vector Database

A database optimized for storing and searching embeddings. It supports semantic search across large sets of documents or records.

Semantic Search

Search based on meaning rather than exact keywords. It uses embeddings to find relevant content even when wording differs.

Relevance

How closely retrieved context matches the user's question or task. Irrelevant context wastes tokens and can make answers worse.

Sufficiency

Whether the provided context contains enough information to answer correctly. A small but complete context is often better than a huge incomplete one.

Structure

How clearly context is organized for the model. Headings, ordering, deduplication, and source labels can improve model performance.

Data Plumbing

The ingestion, cleaning, storage, retrieval, routing, and formatting of data around the model. Many LLM app wins come from better plumbing rather than changing models.

Multimodal AI

Multimodal models can work across text, images, audio, video, and files — enabling agents to inspect richer environments and interact with the world in more human-like ways.

Vision Model

A model that understands images or visual inputs. It can classify images, read screenshots, extract information, describe scenes, or support visual QA.

Vision-Language Model

A model that combines image understanding with text reasoning. Useful for asking questions about screenshots, diagrams, forms, and UI states.

Audio AI

AI systems that process or generate speech and sound. Common use cases include transcription, voice agents, translation, summarization, and real-time speech interaction.

Image Generation

Creating images from text or image prompts. Builders choose models based on style control, realism, prompt following, speed, cost, and licensing.

Multimodal Model

A model that can work across more than one data type — text, images, audio, video, or files. Multimodal agents can inspect richer environments than text-only agents.

Transcription

Converting spoken audio into text. It is often the first step in meeting summaries, call analytics, and voice interfaces.

Text-to-Speech

Generating spoken audio from text. It is used for voice assistants, accessibility, narration, and real-time conversational agents.

Coding with AI

AI coding harnesses turn a model from a code suggester into a coding assistant or agent — connecting it to a codebase, editor, shell, tests, browser, and file system. The human still reviews everything.

AI Coding Harness

A product or system that connects an AI model to a codebase, editor, shell, tests, browser, and file system. It turns a model from a code suggester into a coding assistant or agent.

OpenAI Codex

OpenAI's cloud-based software engineering agent that can read code, edit files, run commands, verify work, and open pull requests. Built on top of reasoning models, not simply an API model.

Cursor

An AI-native code editor focused on code generation, chat, and codebase-aware assistance. Strongest when the developer stays actively involved in the editing loop.

Claude Code

Anthropic's coding agent interface for working with local codebases from the terminal. Part of the broader category of AI coding harnesses.

Windsurf

An AI coding environment designed around codebase context and assisted development. Like other harnesses, its value depends on review, tests, and clear task specs.

Vibe Coding

A casual term for building software by steering AI-generated code through natural language. It can move fast, but the human still needs to review architecture, correctness, security, and maintainability.

Code Review

Reading generated or human-written code to find bugs, bad abstractions, security issues, and maintainability problems. AI makes review more important, not less.

Test

A repeatable check that code behaves as expected. Tests are one of the best verification tools for AI-assisted coding.

Sandbox

An isolated environment where code can run with limited access. Sandboxes reduce risk when agents execute commands or modify files.

Pull Request

A proposed code change submitted for review before merging. Coding agents may open pull requests after completing and verifying a task.

System Design

The practice of choosing architecture, components, data flow, and trade-offs for a software system. AI can suggest implementations, but engineers must judge whether the design is appropriate.

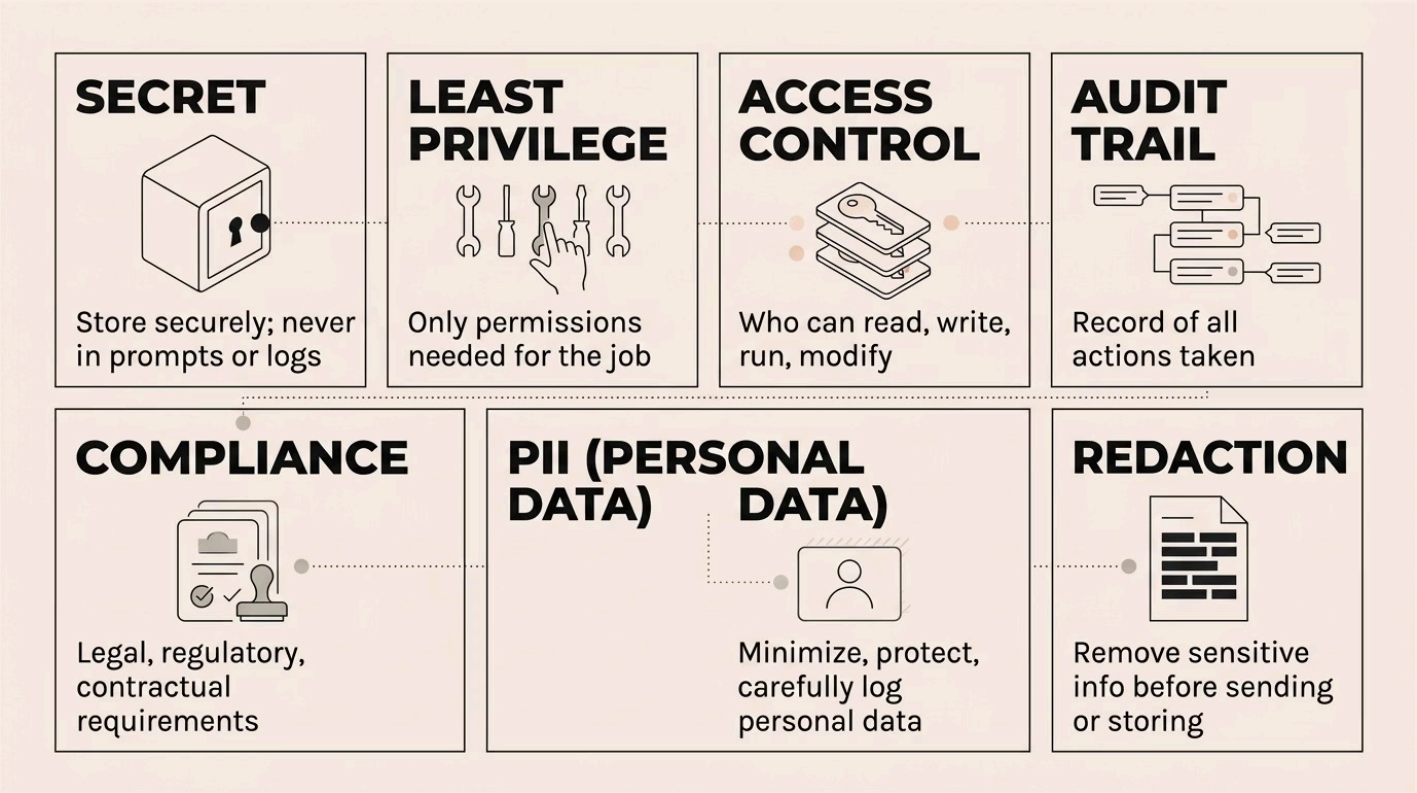
Technical Debt

Future cost created by rushed, brittle, duplicated, or poorly understood code. AI-generated code can create debt quickly if no one reviews or refactors it.

> ⚠ AI makes code review **more important**, not less. Always review generated code for correctness, security, and maintainability.

Security and Governance

Security and governance are not afterthoughts — they shape model choice, data storage, logging, and human review from the start. Agents with broad access require especially careful controls.



Secret

Sensitive information such as API keys, tokens, passwords, or private credentials. Secrets should be stored securely and never placed in prompts, logs, screenshots, or public repositories.

Least Privilege

Giving a system only the permissions it needs to do its job. Agents should not receive broad access when a narrow tool or scope is enough.

Access Control

Rules that decide who or what can read, write, run, or modify resources. Critical when agents interact with private data or production systems.

Audit Trail

A record of actions taken by users, agents, tools, or systems. Audit trails help debug incidents and prove what happened.

Compliance

Meeting legal, regulatory, contractual, or organizational requirements. Model choice, data storage, logging, and human review may all be shaped by compliance.

PII

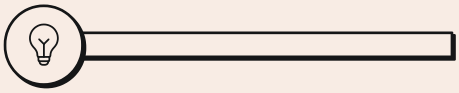
Personally identifiable information — names, emails, addresses, IDs, or phone numbers. LLM apps should minimize, protect, and carefully log PII.

Redaction

Removing or masking sensitive information before sending, storing, or displaying it. Redaction reduces privacy and security risk.

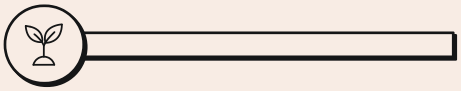
Production Readiness

Production LLM apps need monitoring, fallbacks, cost controls, security, and support paths. A prototype is not a production system — and the gap between them is where most AI projects succeed or fail.



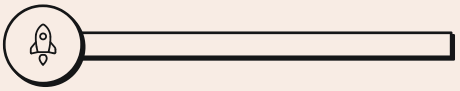
Prototype

An early version built to test feasibility or learning. Prototypes can be messy, but they should not be mistaken for production systems.



MVP

Minimum Viable Product — the smallest version that delivers real value and tests the core assumption. For AI apps, the MVP must include enough evaluation to know whether the model output is usable.



Production

A deployed system used by real users or business workflows. Production LLM apps need monitoring, fallbacks, cost controls, security, and support paths.

SLA

Service Level Agreement — a reliability or performance commitment. If an LLM provider or workflow has an SLA, your app design should account for what happens when it is not met.

Cost Control

The set of practices that reduce LLM spend — caching, routing, shorter prompts, batching, cheaper models, and limiting output tokens. Cost control should be designed early, not added after the bill arrives.

Quality Control

Processes that ensure outputs meet the required standard. For LLM apps this can include schemas, validators, evaluations, human review, and post-generation checks.

User Feedback

Signals from users about whether outputs are useful, correct, or frustrating. Feedback helps improve prompts, routing, evals, and product design.

Benchmark

A standard test used to compare models or systems. Benchmarks are useful starting points, but your own task-specific evals matter more for product decisions.

A/B Test

A comparison where users or metrics determine which of two versions performs better. In AI apps, A/B tests can compare prompts, models, workflows, or UI choices.

Production readiness is a spectrum, not a binary. Start with evaluation, add monitoring, then layer in cost controls, fallbacks, and security as you scale.

Takeaway Terms

Five essential principles that anchor everything in Week 1 — from exploring the model landscape to writing better prompts, managing context, and staying in the reviewer's seat.

1

Builder's Starting Point

The practical first step before building: explore model options, understand costs and constraints, and match the tool to the use case. Week 1 emphasizes exploring before committing to an architecture.

2

Model Landscape

The current ecosystem of providers, open and closed models, model hubs, benchmarks, pricing, and hosting options. Builders need enough landscape awareness to choose wisely.

3

Prompt Quality In, Quality Out

The idea that vague prompts usually produce vague outputs. Treat prompts like mini specs when you want dependable results.

4

One Task per Conversation

A practical coding guideline to prevent context pollution. Fresh contexts help the model focus on the current goal instead of stale history.

5

You Are Still the Reviewer

The central rule for AI-assisted coding and building. The model can generate work, but the human remains responsible for correctness, security, and product judgment.

> **The model can generate work, but the human remains responsible for correctness, security, and product judgment.** This is the most important principle to carry forward from Week 1 into everything you build.